

Архитектурные паттерны: RAG и его продвинутые вариации

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Фомин Дмитрий

Руководитель отдела архитектуры

- Больше 15 лет опыта работы архитектором
- Участие в разработке архитектуры в очень больших проектах (BSS)

 @advoc_diaboly

 fomin.dmitry@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#OTUS**
AI-Architect-2025-12



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент
и планирование проекта

Проектирование и
документирование архитектуры

Продвинутые
архитектурные
паттерны

Инфраструктура

Качество, интеграции
и безопасность

Стратегия, лидерство и
экономика

Проектная работа

ФИНИШ



Маршрут вебинара

Знакомство

Постановка задачи

Анатомия базового RAG

Проблемы базового RAG

Advanced RAG

Основные тезисы

Цели вебинара

К концу занятия вы сможете

проектировать и реализовывать
системы на основе паттерна
Retrieval-Augmented Generation (RAG);

анализировать и применять
продвинутые техники RAG для
повышения качества ответов.

Смысл

Для чего вам это уметь

Чтобы LLM использовала релевантную
информацию при подготовке ответа

Постановка задачи

LLM — это «Замороженный мозг»

- Параметрическая память (Parametric Knowledge): Знания, зашитые в весах модели во время обучения.
 - Статична: Не меняется после тренировки.
 - Устаревает: Не знает новостей, курсов валют, новых библиотек.
- Knowledge Cutoff: Дата отсечки знаний (например, для Qwen 2.5 — это конец 2023 / начало 2024).
-
- Проблема: Модель — это отличный «процессор» для логики, но плохой «жесткий диск» для актуальных фактов.



Почему нельзя обучить модель на актуальных данных?

- Pre-training (Обучение с нуля)
 - Цель: Научить модель языку, логике и общим знаниям о мире.
- Данные: Триллионы токенов (The Whole Internet).
- Ресурсы: Тысячи GPU (H100), месяцы работы.
- Стоимость: \$10M – \$100M+.



Почему нельзя дообучить модель на актуальных данных?

- Fine-Tuning (Дообучение)
 - Цель: Адаптировать стиль, формат ответа или специфический жаргон (Medical, Legal).
 - Ограничение: Не подходит для внедрения новых фактов.
 - Проблема: "Catastrophic Forgetting" (модель забывает старое, пытаясь вызубрить новое).
- Стоимость: Тысячи долларов.

Почему нельзя дообучить модель на актуальных данных?

- RAG (In-Context Learning)
 - Цель: Предоставить актуальные факты здесь и сейчас.
- Стоимость: \$0 на обучение (только инференс и хранение).
- Обновление знаний: Мгновенно (INSERT в Vector DB).



Галлюцинации и уверенная ложь

- Природа галлюцинаций: Модель — это вероятностный генератор следующего токена, а не база знаний.
- Цель модели: Сгенерировать правдоподобный (связный) текст, а не истинный.
- Confidence vs Accuracy:
 - LLM звучит уверенно даже когда ошибается.
 - Отсутствие данных -> Генерация синтетического ответа.
- Риск для бизнеса: Принятие решений на основе выдуманных фактов.

Почему не просто Long Context?

- Контекстные окна растут: 128k, 200k, 1M+ токенов. Зачем тогда RAG?
- Проблема 1: Lost in the Middle.
 - Модели хуже находят информацию в середине длинного промпта.
- Проблема 2: Стоимость (Cost).
 - Отправлять всю базу знаний (книгу, документацию) в каждом запросе — сжигание денег.
- Проблема 3: Latency.
 - Обработка 100k токенов на вход занимает секунды (Time to First Token).



Вопросы



если есть вопросы



если вопросов нет

Анатомия базового RAG

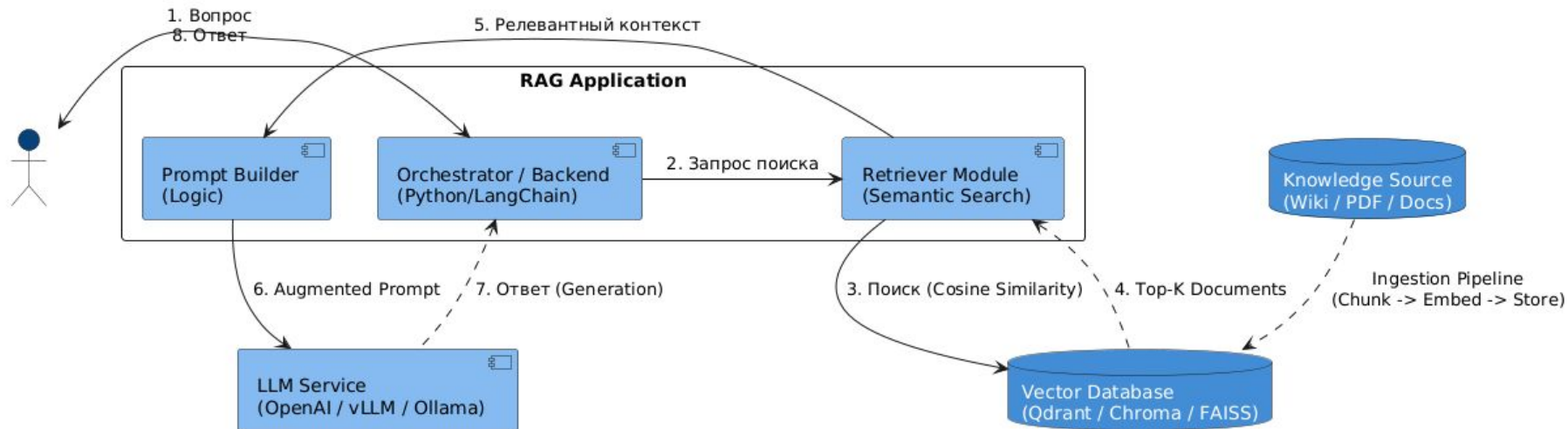
Анатомия базового RAG

- RAG (Retrieval-Augmented Generation):
 - Паттерн проектирования, объединяющий генеративные возможности LLM с внешним поиском.
- Non-Parametric Memory:
 - Внешний индекс (Vector DB, SQL, Graph), который можно обновлять в реальном времени.
- Разделение ответственности:
 - Retriever: Ищет факты (Актуальность).
 - Generator (LLM): Формулирует ответ (Рассуждение).



High-Level архитектура RAG

High-Level Architecture: RAG System



Этап 1: Ingestion Pipeline. Chunk

1. LLM имеет лимит контекста, а поиск должен быть точным.
 - a. Нельзя подавать "Войну и мир" целиком.
2. Стратегии:
 - a. Fixed Size: Жестко по 2000 символов (быстро, но рвет смысл).
 - b. Recursive: Умная нарезка по абзацам/предложениям.
 - c. Semantic: Нарезка по смыслу (advanced).

```
# Данные (AG News)
dataset = load_dataset("ag_news", split="train")
docs = dataset['text'][:2000] # Берем 2000 символов для теста
```

Этап 1: Ingestion Pipeline. Embed

1. Embedding Model: Переводит текст в массив чисел (вектор).
 - a. Модель all-MiniLM-L6-v2:
 - i. Легкая (80MB), быстрая, идеальна для CPU/Local RAG.
 - ii. Размерность: 384 измерения.
2. Семантическая близость:
 - a. Вектор фразы "Ищу ноутбук" будет ближе к "Купить компьютер", чем к "Рецепт пирога", даже если общих слов нет.

```
# Загружаем модель для эмбедингов
embedder = SentenceTransformer('all-MiniLM-L6-v2', device=DEVICE)
# Создаем эмбединги
# Если DEVICE='cpu', это займет чуть больше времени, чем на RTX
doc_embeddings = embedder.encode(docs, convert_to_numpy=True,
show_progress_bar=True)
```



Этап 1: Ingestion Pipeline. Store

1. FAISS (Facebook AI Similarity Search):
 - a. Библиотека для быстрого поиска ближайших соседей.
 - b. IndexFlatL2: Точный поиск (Brute-force) по Евклидову расстоянию.
2. Production vs Prototype:
 - a. Notebook: FAISS в памяти (RAM).
 - b. Prod: Qdrant, Weaviate, PGvector (Persistence, Filtering, Scaling).
3. Почему L2 (Евклид), а не Cosine?
 - a. Для нормированных векторов это математически эквивалентно ранжированию.

```
d = doc_embeddings.shape[1] # Получаем размерность эмбедингов
index = faiss.IndexFlatL2(d) # Создаем простой L2-индекс (поиск по Евклидову
расстоянию)
index.add(doc_embeddings)    # Добавляем все наши векторы в индекс
```



Этап 2: Retriever Module

1. Запрос пользователя векторизуется той же моделью (embedder).
 - а. Ищем top-k ($k=3$) ближайших соседей.

```
def search(query, k=3):
    query_vec = embedder.encode([query]) # Векторизуем на том же устройстве
    distances, indices = index.search(query_vec, k)
    return [docs[idx] for idx in indices[0]]

def define_context(user_query):
    found_docs = search(user_query, k=3)      # ШАГ А: Достаем факты из базы
    context_str = "" # ШАГ Б: Формируем контекст (Склеиваем новости в одну
строку)
    for i, doc in enumerate(found_docs):
        context_str += f"Fact {i+1}: {doc}\n"
    return context_str, found_docs
```



Этап 3: LLM Service

1. Чистая специфика конкретной модели. К RAG имеет мало отношения

```
def generate_rag_answer(prompt):
    messages = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": prompt}
    ] # 1. Формируем шаблон чата
    text = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    inputs = tokenizer([text], return_tensors="pt").to(DEVICE) # 2. Токенизация с явным указанием возврата маски
    with torch.no_grad():
        generated_ids = model.generate(
            input_ids=inputs.input_ids,
            attention_mask=inputs.attention_mask,
            pad_token_id=tokenizer.eos_token_id,
            max_new_tokens=200,
            temperature=0.7,
            do_sample=True # Включаем семплирование для разнообразия
        )
    generated_ids = [
        output_ids[len(input_ids):] for input_ids, output_ids in zip(inputs.input_ids, generated_ids)
    ] # 4. Декодирование (обрезаем входной промпт)
    response = tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
    return response
```



Этап 3: Prompt Builder

1. "Инженерный промпт": Мы жестко ограничиваем модель (Answer based ONLY on context).
 - а. Вставляем найденные факты прямо в текст запроса.

```
def ask_rag(user_query):
    context_str, found_docs = define_context(user_query)
    rag_prompt = f"""You are an analyst. Answer the question based ONLY on the
provided context facts.
If the answer is not in the context, say "I don't know based on these facts".
--- CONTEXT START ---
{context_str}
--- CONTEXT END ---
Question: {user_query}
""" # ШАГ В: Создаем "Инженерный Промпт"
    answer = generate_rag_answer(rag_prompt) # ШАГ Г: Отправляем это в LLM
    (используем вашу исправленную функцию)
    return answer, found_docs, rag_prompt
```



Демонстрация: Вопрос модели без контекста



Первый вопрос: Who defeated the Rockets?



Ищу информацию по запросу: 'Who defeated the Rockets?'...



Генерирую ответ...



Ответ RAG:

Based solely on the provided context, I do not know who defeated the Rockets. The context does not mention any specific defeat involving the Rockets or any other team named "Rockets". Therefore, my response is:

I don't know based on these facts.



Демонстрация: Вопрос модели без контекста

 Добавляю знание: 'The Detroit Pistons defeated the Houston Rockets 87-79 to open the NBA season as defending champions.'

 База знаний обновлена. Всего документов: 2001

 Повторный вопрос: Who defeated the Rockets?

 Ищу информацию по запросу: 'Who defeated the Rockets?'...

 Генерирую ответ...

 Ответ RAG:

The Detroit Pistons defeated the Houston Rockets.

```
missing_news = "The Detroit Pistons defeated the Houston Rockets 87-79 to open the  
NBA season as defending champions." # 1. Добавляем "Золотую запись" (ту самую  
новость) в наши документы
```

```
print(f" Добавляю знание: '{missing_news}')
```

```
docs.append(missing_news) # Добавляем текст в список документов (в конец)
```

```
new_embedding = embedder.encode([missing_news]) # 2. Создаем для нее эмбединг и  
обновляем индекс
```

```
index.add(new_embedding)
```

Вопросы



если есть вопросы



если вопросов нет

Проблемы базового RAG

Проблема 1: Bad Retrieval

- Семантический поиск не идеален:
 - "Почти правильно" для поисковика — это "Галлюцинация" для RAG.
- Слепые зоны Dense Retrieval:
 - Точные совпадения: Артикулы (SKU), ID транзакций, версии ПО (v1.2 vs v1.2.1).
- Negation (Отрицание): Фразы "не удалять" и "удалять" имеют почти одинаковый вектор (косинус > 0.9), но противоположный смысл.

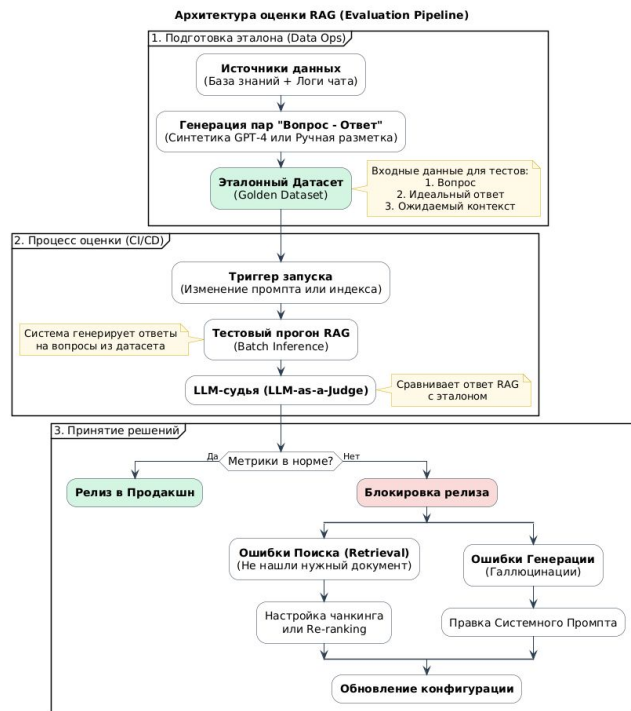
Проблема 2: Потеря контекста

- Top-K Dilemma:
 - Мало ($k=1$): Риск пропустить ответ.
 - Много ($k=10$): Context Pollution.
- Lost in the Middle (снова):
 - Если правильный ответ зажат между двумя нерелевантными документами, модель может его проигнорировать.
- Distraction:
 - Противоречивая информация в разных чанках сбивает модель с толку (LLM пытается "усреднить" правду).

Проблема 3: Игнорирование инструкций

- Strong Priors (Сильные предубеждения):
 - Модели обучались на триллионах токенов. Их "уверенность" в своих знаниях часто перевешивает те 5 строчек текста, что вы подали в контексте.
- Пример:
 - Вы подаете в контекст: "Земля плоская (согласно документу X)".
 - Модель отвечает: "Земля круглая", игнорируя инструкцию, потому что её веса "знают" правду.
- Слабые модели (7B и меньше) часто забывают инструкцию "Answer ONLY based on context", когда контекст противоречит их знаниям.

Метрики качества RAG



- Как мы измеряем качество? "На глаз" не работает.
- Framework RAGAS (RAG Assessment):
 - Faithfulness: Насколько ответ модели соответствует контексту (нет ли галлюцинаций).
 - Context Precision: Насколько найденные документы релевантны вопросу (работа Retriever).
 - Answer Relevance: Насколько ответ полезен пользователю.
- Для тестов нужен набор эталонных вопросов и ответов (Golden Dataset).
- RAGAS использует другую LLM для оценки (LLM-as-a-judge). Это стоит денег и может быть медленно». Это важный архитектурный нюанс.

Вопросы



если есть вопросы



если вопросов нет

Advanced RAG — Улучшаем Поиск

Гибридный поиск (Hybrid Search)

- Dense (Вектора): Понимают смысл, синонимы, опечатки. ("Computer" найдет "Laptop").
- Sparse (BM25 / Keywords): Точное совпадение слов, аббревиатур, артикулов.
- Hybrid Search:
 - Используем оба метода параллельно.
 - Нормализуем скоры (Reciprocal Rank Fusion - RRF).
- Получаем надежный результат даже для специфических запросов.



Реализация гибридного поиска

```
# В продакшене тут нужен нормальный лемматизатор, для демо хватит split()
tokenized_corpus = [doc.lower().split() for doc in demo_docs] # 1. Токенизация для
BM25 (разбиваем текст на слова)
bm25 = BM25Okapi(tokenized_corpus) # 2. Создание индекса BM25

def search_hybrid(query, k=5):
    query_tokens = query.lower().split() # А. Поиск по ключевым словам (BM25)
    bm25_docs = bm25.get_top_n(query_tokens, demo_docs, n=k) # Получаем top-k
    документов (возвращает сами тексты)
    vector_docs = search(query, k) # Б. Векторный поиск (FAISS)
    # В. Объединение (Union)
    # Используем set для удаления дубликатов, сохраняя порядок через list
    combined_docs = list(set(bm25_docs + vector_docs))
    print(f" 🔍 BM25 нашел: {len(bm25_docs)} | Vector нашел: {len(vector_docs)}")
    print(f" ∑ После объединения: {len(combined_docs)} уникальных кандидатов")

    return combined_docs
```

Post-Retrieval: Re-ranking

- Two-Stage Retrieval (Двухэтапный поиск):
 - Stage 1: Bi-Encoder (Retriever).
 - Быстро, но грубо. Сравнивает вектора по отдельности.
 - Stage 2: Cross-Encoder (Reranker).
 - Как работает: Принимает на вход ПАРУ (Вопрос + Документ) и выдает скор релевантности (0..1).
 - Точность: Гораздо выше, видит логические связи.
 - Минус: Медленный (нельзя прогнать по всей базе), поэтому применяем только к Top-50.



Реализация Re-ranking.

```
def define_reranked_context(user_query, initial_k, final_k):
    # 1. Сначала достаём МНОГО кандидатов (Initial Retrieval)
    candidates = search(user_query, k=initial_k)
    pairs = [[user_query, doc] for doc in candidates]      # Готовим пары для модели:
    [[Query, Doc1], [Query, Doc2], ...]
    scores = reranker.predict(pairs)      # Получаем оценки (scores)
    scored_docs = sorted(zip(candidates, scores), key=lambda x: x[1], reverse=True)
    # Соединяем документы с их оценками и сортируем
    # 3. Вывод результатов реранкинга (для демонстрации)
    for i, (doc, score) in enumerate(scored_docs[:5]): # Показываем топ-5
        print(f"Rank {i+1} (Score: {score:.2f}): {doc[:80]}...")
    final_docs = [doc for doc, score in scored_docs[:final_k]]      # 4. Отбираем
    лучшие для LLM
    context_str = ""      # 5. Генерация (как раньше)
    for i, doc in enumerate(final_docs):
        context_str += f"Fact {i+1}: {doc}\n"

    return context_str, final_docs
```



Демонстрация: Попробуем сломать модель



Первый вопрос: Why did apple fall?



Ищу информацию по запросу: 'Why did apple fall?'...



Генерирую ответ...



Ответ RAG:

I don't know based on these facts.



RAG prompt:

You are an analyst. Answer the question based ONLY on the provided context facts.

If the answer is not in the context, say "I don't know based on these facts".

--- CONTEXT START ---

Fact 1: Isaac Newton sat under a tree and watched an apple fall to the ground.

Fact 2: Market reports indicate that poor iPhone sales led to a significant stock decline.

Fact 3: Rival Targets Apple's iTunes Music Customers By ALEX VEIGA LOS ANGELES (AP) -- For more than a year, Apple Computers Inc. enjoyed singular success selling songs exclusively to users of its iPod portable music player...

--- CONTEXT END ---

Question: Why did apple fall?



Демонстрация: А что с реранкингом?

🗣️ Первый вопрос: Why did apple fall?

--- Результаты переранжирования ---

Rank 1 (Score: 0.96): Market reports indicate that poor iPhone sales led to a significant stock declin...

Rank 2 (Score: 0.01): Real to Apple: It #39;s On An old-fashioned price war is gearing up in the digit...

Rank 3 (Score: 0.01): Apple #39;s Real Rivalry If you go by the headlines alone, it sounds like RealNe...

Rank 4 (Score: 0.01): RealNetworks Tries to Core Apple (washingtonpost.com) washingtonpost.com - RealN...

Rank 5 (Score: 0.00): Rival Targets Apple's iTunes Customers For more than a year, Apple Computers Inc...

🤖 [3] Генерация ответа LLM...

💬 Ответ RAG:

Apple fell due to poor iPhone sales, which led to a significant stock decline according to market reports.

💬 RAG prompt:

You are an analyst. Answer based ONLY on context facts.

--- CONTEXT START ---

Fact 1: Market reports indicate that poor iPhone sales led to a significant stock decline.

Fact 2: Real to Apple: It #39;s On An old-fashioned price war is gearing up in the digital music download business, with RealNetworks (Quote, Chart) on the offensive. With rival Apple in its crosshairs, RealNetworks has slashed the price on songs sold from its RealPlayer store by 50 ...

Fact 3: Apple #39;s Real Rivalry If you go by the headlines alone, it sounds like RealNetworks (Nasdaq: RNWK) is about to embark on a heady money-losing venture as it halves prices for its music download service Harmony, looking to take a bite out of Apple ...

--- CONTEXT END ---

Question: Why did apple fall?

Metadata Filtering

- Проблема:
 - Пользователь спрашивает "Мои штрафы". Вектор находит штрафы других людей, так как они семантически похожи.
- Решение: Pre-filtering.
 - Жесткие фильтры по метаданным применяются до или во время векторного поиска.

Pre-Retrieval Optimization.

- Проблема: Пользователи задают короткие, нечеткие вопросы («почему не работает», «где договор»). Векторный поиск по таким запросам работает плохо.
- Query Expansion (Расширение запроса):
 - LLM генерирует синонимы или похожие вопросы.
- Query Rewriting (Переписывание):
 - LLM переписывает вопрос, добавляя контекст диалога.
 - Было: "А сколько он стоит?" (непонятно кто).
 - Стало: "Какова цена ноутбука Macbook Pro, который мы обсуждали?"
- HyDE (Hypothetical Document Embeddings):
 - LLM пишет гипотетический идеальный ответ, и мы ищем векторную близость к этому ответу, а не к вопросу.



Вопросы



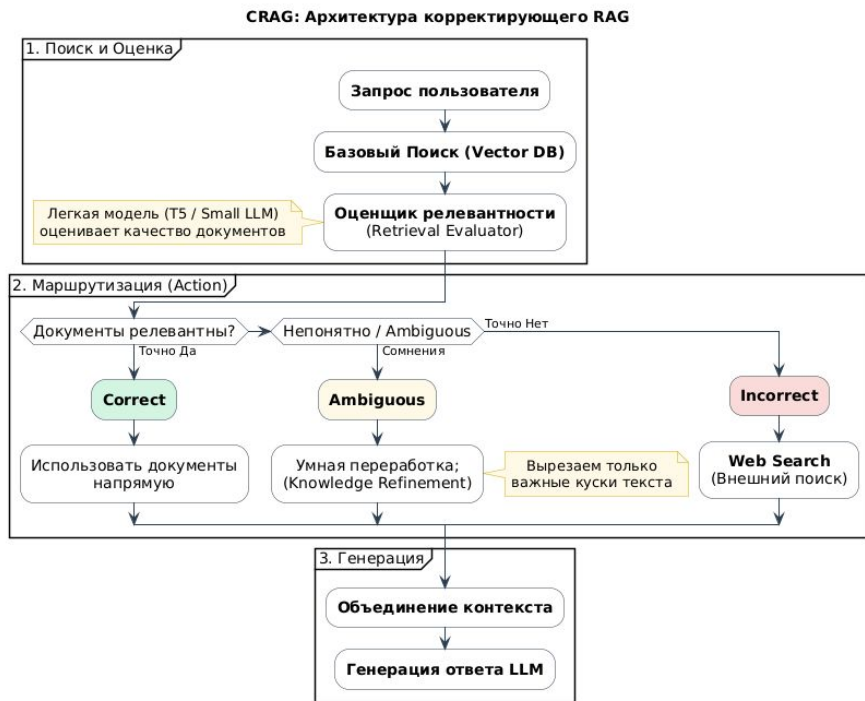
если есть вопросы



если вопросов нет

Advanced RAG — Улучшаем Генерацию и Пайплайн

CRAG (Corrective RAG)



- Проблема: Векторный поиск иногда возвращает нерелевантные документы (мусор), и LLM начинает галлюцинировать на их основе.
- Решение: Внедрить легковесную модель-оценщика (Evaluator) после поиска, но перед генерацией.
- Алгоритм:
 - Получаем документы.
 - Оцениваем их релевантность (Confidence Score).
 - Выбираем стратегию (Action) в зависимости от оценки.

Self-RAG (Self-Reflective RAG)



- В отличие от базового RAG, здесь внедрен цикл обратной связи.
- Модель выступает в роли критика (Critic) по двум критериям:
 - Релевантен ли найденный документ запросу?
 - Подтверждается ли сгенерированный ответ найденными фактами?
- Дифференцированная стратегия исправления:
- Если проблема в данных → переписываем запрос и ищем снова.
- Если проблема в логике → просим модель регенерировать ответ строже).
- Снижение уровня галлюцинаций за счет возможности сказать «Я не знаю», если цикл исчерпал лимит попыток.

Knowledge Graphs (GraphRAG)

- Вопрос: "Как связаны CEO компании A и инвестор компании B?"
 - Вектор: Находит статью про CEO и статью про Инвестора. Связи не видит.
 - Граф: Видит цепочку: CEO --(работал в)--> Company X --(владеет)--> Investor.
- Ищем не похожесть, а связи
- Схема
 - Documents -> LLM Extraction -> Knowledge Graph
 - LLM читает текст и извлекает триплеты: (Apple) --[competitor]--> (Microsoft).
 - Neo4j, Microsoft GraphRAG, LangChain Graph
 - Query -> Graph Traversal -> Sub-graph
 - Sub-graph -> Context for Answer

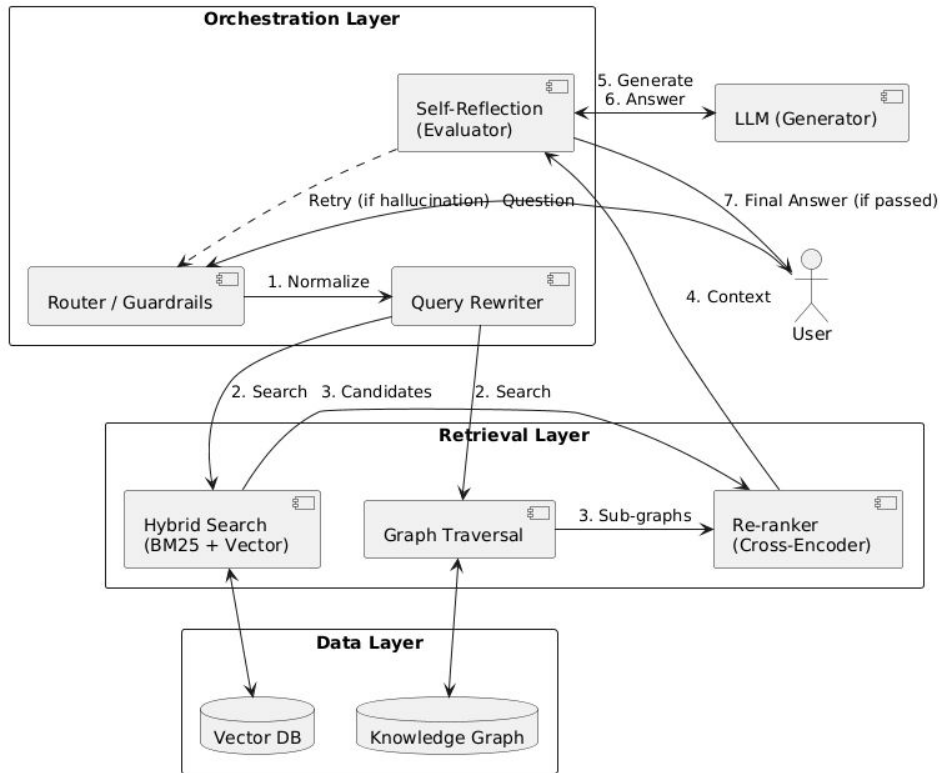
Cache Augmented Generation

- Проблема
 - Принцип Парето: 20% вопросов задаются 80% времени (например, «Как оформить отпуск?», «График работы»).
 - Двойная трата: Каждый раз мы тратим деньги на Embedding + Vector Search + LLM Generation для одного и того же ответа.
- Решение: Semantic Cache (Семантический кэш)
 - В отличие от обычного кэша (Redis get(key)), который ищет точное совпадение текста, семантический кэш ищет похожие по смыслу вопросы.
- Инструменты:
 - GPTCache (популярная библиотека).
 - Redis / Qdrant (как хранилище векторов кэша).



Production RAG Architecture

Production RAG Architecture 2025



- Orchestration Layer (Мозг системы):
 - Внедрение Router & Guardrails для фильтрации вредоносных или нецелевых запросов до начала поиска.
 - Query Rewriter оптимизирует "человеческий" запрос
- Retrieval Layer
 - Комбинация трех стратегий: Vector Search (смысл) + BM25 (ключевые слова) + Knowledge Graph (связи).
 - Использование Re-ranker (Cross-Encoder) для пересортировки кандидатов.
- Модуль Self-Reflection оценивает ответ перед отправкой пользователю. При галлюцинациях запускается цикл регенерации (Retry).

Вопросы



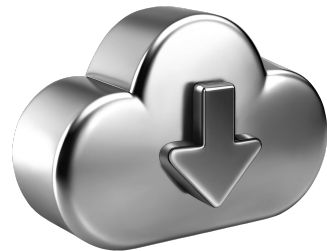
если есть вопросы



если вопросов нет

Список материалов для изучения

1. Вейс, Энди, и Бен Аффало. 2024. Проектирование и создание приложений с большими языковыми моделями (LLM). СПб.: БХВ-Петербург.
2. Хуен, Чип. 2023. Проектирование систем машинного обучения. Итеративный процесс для создания готовых к работе приложений. СПб.: БХВ-Петербург.
3. Лакшманан, Валлаиаппа, и др. 2022. Паттерны проектирования для машинного обучения. СПб.: Питер.
4. Mabrouk, M., and F. Z. Belouadha. 2024. Vector Databases: A Comprehensive Guide. Apress.
5. Labonne, Maxime. 2024. The LLM Practical Guide: From Training to Deployment. O'Reilly Media.
6. Ozdemir, Sinan. 2023. Practical Large Language Models: A Hands-On Guide to Building Real-World LLM Applications. Addison-Wesley Professional.



**Подведём
итоги занятия**

Теперь вы сможете:

Отправьте в чат номера достигнутых целей

проектировать и реализовывать
системы на основе паттерна
Retrieval-Augmented Generation (RAG);

анализировать и применять
продвинутое техники RAG для
повышения качества ответов.

Чтобы LLM использовала релевантную
информацию при подготовке ответа

Вопросы для проверки

1. Зачем нам RAG, если у Gemini/GPT-4 контекст 1-2 миллиона токенов?
Можно же просто загрузить всю книгу.
2. Re-ranking замедляет систему. Как бороться с Latency?
3. Как гарантировать, что модель не будет галлюцинировать (врать)?
4. Когда стоит использовать Fine-tuning вместо RAG?

Ключевые тезисы занятия

1. Naive RAG — это прототип, а не продакшн.
2. Hybrid Search + Re-ranking = Новый стандарт
3. Качество данных важнее качества модели



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

